

Editor de texto colaborativo com Consistência Eventual

Maykon Marcos Júnior (22102199)
Gabriel Turatti Andrade (21105479)
Tális Breda (22102202)

Universidade Federal de Santa Catarina (UFSC)
INE5418 - Computação distribuída

1 de dezembro de 2025

1 Decisões de implementação

1.1 Languages e tecnologias

A aplicação foi construída utilizando Rust, escolhida por conta da experiência do grupo com a linguagem, que foi a mesma utilizada no trabalho 1 da disciplina.

1.2 Estratégias

O grupo de nodos é criado de forma fixa, antes da execução dos processos, a mesh de conexões Peer to Peer é inicializada, garantindo que todos os nodos tenham os endereços de todos os outros nodos, para posteriormente criar as conexões.

O arquivo sendo alterado pelos nodos é representado por um vetor de ColabChars, cada Colabchar possui um vetor relógio indicando sua posição em relação aos outros. Se existem 5 caracteres seguidos com relógios (3.1) (3.2) (3.3) (3.4) e (3.5), e um novo caractere é inserido na posição correspondente ao (3.2), o arquivo agora terá caracteres com relógios (3.1) (3.2) (3.2.1) (3.3) (3.4) e (3.5).

Isso é feito para manter a consistência entre diferentes nodos alterando o mesmo arquivo. Quando duas pessoas alteram ao mesmo tempo, os relógios garantem que as posições relativas sejam respeitadas. Caso as duas alterem a posição correspondente com o mesmo relógio, um desempate é feito baseado no ID do nodo, dessa forma ambos podem chegar na mesma conclusão consistentemente.

Por exemplo, caso o Nodo 2 insira (3.2.1) e o Nodo 5 também insira (3.2.1), inicialmente, assumindo que a mensagem do Nodo 5 chegue primeiro, os outros nodos irão colocar o caractere do Nodo 5 na posição (3.2.1). Depois, quando a mensagem do Nodo 2 chegar, o caractere dele irá substituir a posição (3.2.1) pelo critério de desempate do id mais baixo, e o caractere antigo terá seu relógio aumentado para (3.2.1.1), com esse relógio, novamente será feita uma tentativa de escrever ele no arquivo, caso não seja possível, o relógio irá aumentar de precisão até que seja possível.

As alterações de um nodo são feitas primeiro localmente, e depois uma mensagem contendo o

caractere e o relógio é enviada para todos os outros nodos do grupo. Cada nodo, quando recebe uma mensagem, aplica as alterações através da função `merge`, que irá analisar se aquela mensagem é de deleção ou inserção, fazendo as alterações correspondentes.

2 Compilação e execução

O projeto é compilado utilizando `cargo`, que está presente na instalação padrão do Rust (via <https://rust-lang.org/tools/install/>).

Para compilar, basta executar o comando `cargo build`.

3 Exemplos de saída

Foram implementados dois testes principais, que são de inserção concorrente e inserção/deleção concorrente.

3.1 Inserção concorrente

Nesse teste, 5 nodos, após aguardarem um segundo até todas as threads iniciarem, inserem no arquivo o caractere equivalente ao seu ID um total de 10 vezes, em posições sequenciais. Isso testa a capacidade do programa de lidar com a concorrência de inserções, e de garantir a ordem e a consistência entre os nodos.

Não foi possível garantir que todas as mensagens fossem entregues para todos os nodos, mas há garantia de que as mensagens que são entregues para diferentes nodos sempre são postas na mesma ordem. Isso é, se dois nodos recebem as mesmas mensagens, essas mensagens serão postas na mesma ordem relativa entre elas. Alguns nodos acabam não conseguindo receber as mensagens, e esses acabam com uma string diferente no arquivo.

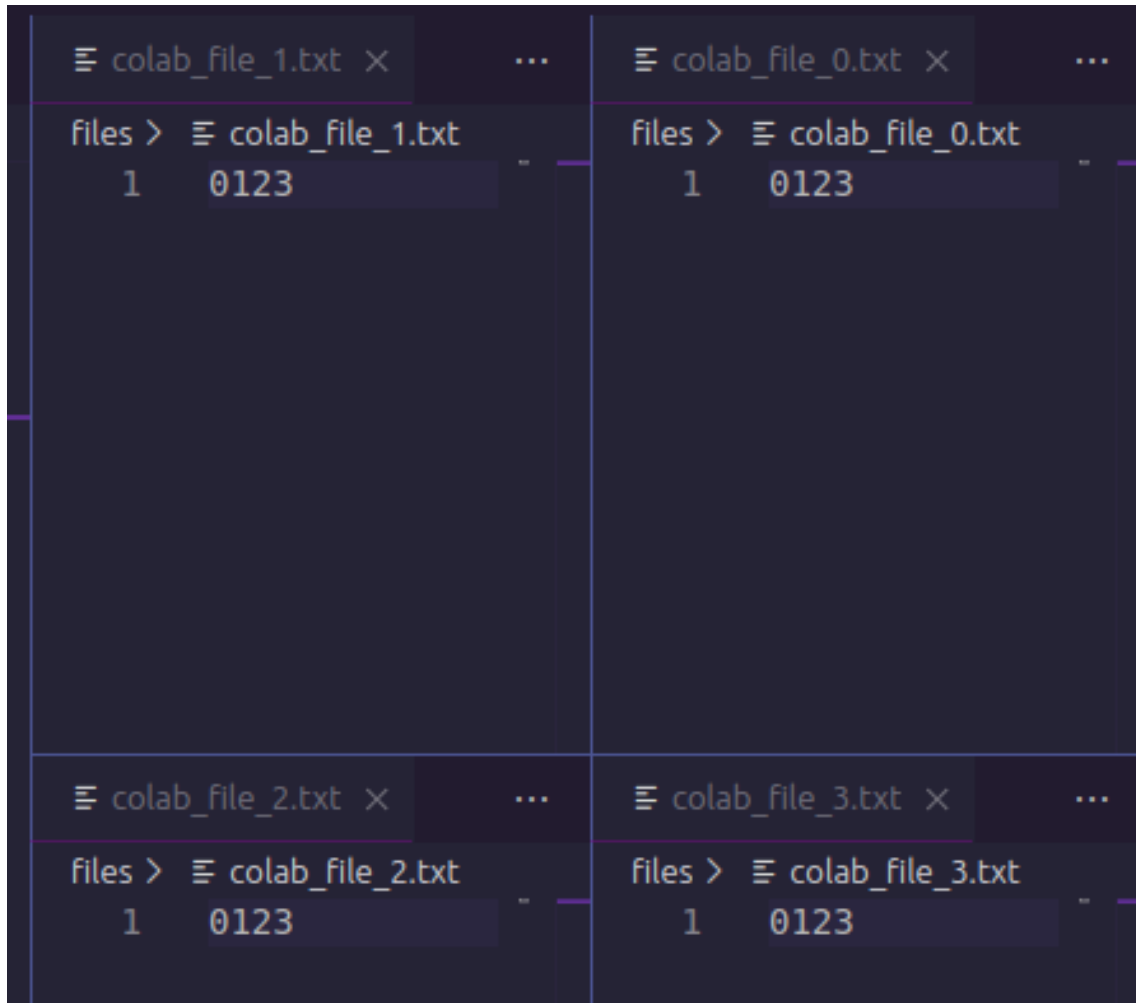


Figura 1: Resultados do teste 1

3.2 Inserção/deleção concorrente

Nesse teste, 8 nodos inserem um único caractere na posição 0 e, depois de aguardarem 1 segundo, os nodos com ID par deletam o caractere da posição 0, e os nodos com ID ímpar inserem um caractere equivalente ao seu ID na posição 1. Isso testa a capacidade do programa de lidar com deleções concorrentes e alterações nas posições dos caracteres.

Quando o ID de um nodo passa de 9, o programa passa a escrever o caractere hexadecimal relacionado ao seu ID. Por exemplo, o programa de ID 11 vai escrever o caractere 'b'.

Assim como no primeiro teste, apesar da ordem ser garantida (se um nodo recebe uma mensagem, ele a recebe na ordem correta), não há garantia de que todas as mensagens são entregues. Isso pode ser visualizado nos resultados do teste, mostrados abaixo:

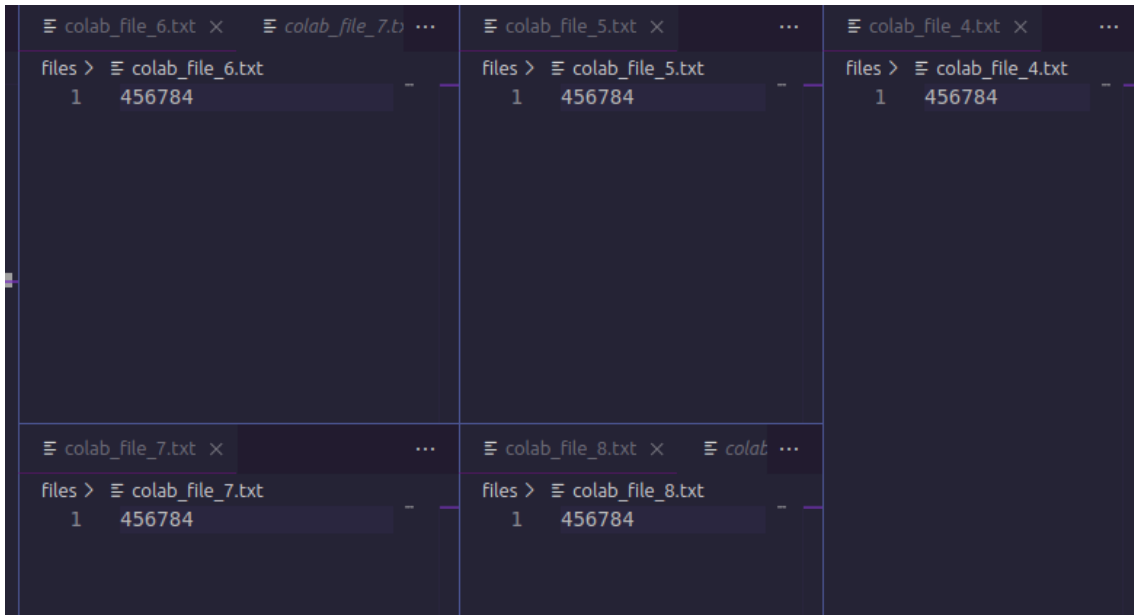


Figura 2: Resultados do teste 2

É possível ver a inconsistência dos envios de mensagem no teste 3, vários nodos não receberam as mesmas mensagens. E apenas o nodo 12 recebeu os dois caracteres 9, no entanto, a consistência é sempre mantida entre os caracteres em comum que os nodos receberam.

Por exemplo, os caracteres "bacd" foram recebidos por todos os 7 nodos, e a ordem é mantida. A ordem dos caracteres "efg9b" também é mantida para todos os 5 nodos que receberam as mensagens.

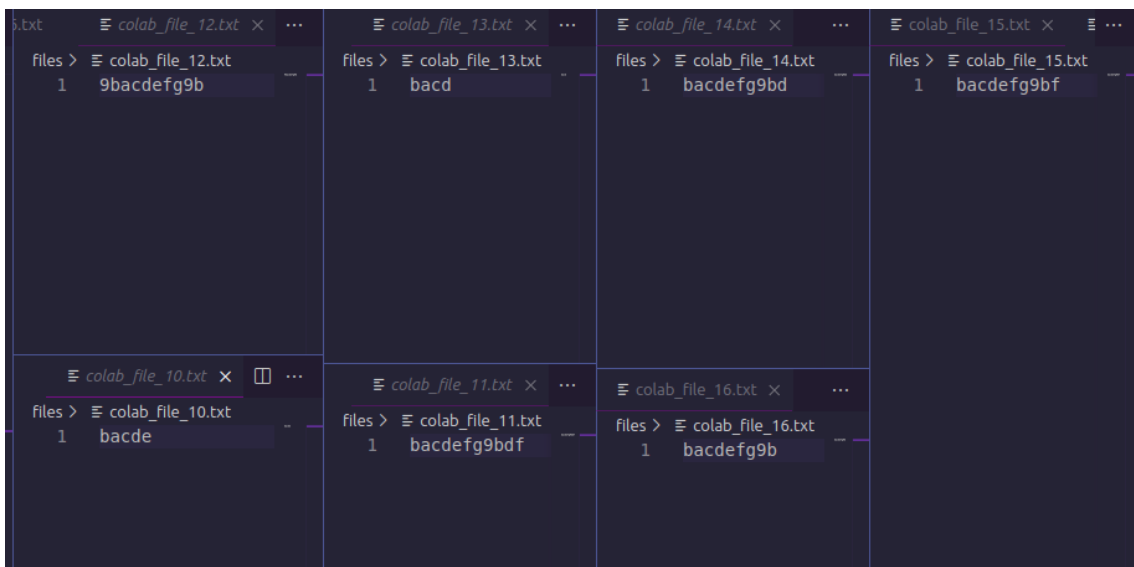


Figura 3: Resultados do teste 3